

otherwise. The first case only happens in the most severe case of sparsity: when a gradient has been zero at all timesteps except at the current timestep. For less sparse cases, the effective stepsize will be smaller. When $(1 - \beta_1) = \sqrt{1 - \beta_2}$ we have that $|\hat{m}_t/\sqrt{\hat{v}_t}| < 1$ therefore $|\Delta_t| < \alpha$. In more common scenarios, we will have that $\hat{m}_t/\sqrt{\hat{v}_t} \approx \pm 1$ since $|\mathbb{E}[g]/\sqrt{\mathbb{E}[g^2]}| \leq 1$. The effective magnitude of the steps taken in parameter space at each timestep are approximately bounded by the stepsize setting α , i.e., $|\Delta_t| \lesssim \alpha$. This can be understood as establishing a *trust region* around the current parameter value, beyond which the current gradient estimate does not provide sufficient information. This typically makes it relatively easy to know the right scale of α in advance. For many machine learning models, for instance, we often know in advance that good optima are with high probability within some set region in parameter space; it is not uncommon, for example, to have a prior distribution over the parameters. Since α sets (an upper bound of) the magnitude of steps in parameter space, we can often deduce the right order of magnitude of α such that optima can be reached from θ_0 within some number of iterations. With a slight abuse of terminology, we will call the ratio $\hat{m}_t/\sqrt{\hat{v}_t}$ the *signal-to-noise ratio (SNR)*. With a smaller SNR the effective stepsize Δ_t will be closer to zero. This is a desirable property, since a smaller SNR means that there is greater uncertainty about whether the direction of $\hat{m}_t$ corresponds to the direction of the true gradient. For example, the SNR value typically becomes closer to 0 towards an optimum, leading to smaller effective steps in parameter space: a form of automatic annealing. The effective stepsize Δ_t is also invariant to the scale of the gradients; rescaling the gradients g with factor c will scale $\hat{m}_t$ with a factor c and $\hat{v}_t$ with a factor c^2 , which cancel out: $(c \cdot \hat{m}_t)/(\sqrt{c^2 \cdot \hat{v}_t}) = \hat{m}_t/\sqrt{\hat{v}_t}$.

3 INITIALIZATION BIAS CORRECTION

As explained in section 2, Adam utilizes initialization bias correction terms. We will here derive the term for the second moment estimate; the derivation for the first moment estimate is completely analogous. Let g be the gradient of the stochastic objective f , and we wish to estimate its second raw moment (uncentered variance) using an exponential moving average of the squared gradient, with decay rate β_2 . Let $g_1, \dots, g_T$ be the gradients at subsequent timesteps, each a draw from an underlying gradient distribution $g_t \sim p(g_t)$. Let us initialize the exponential moving average as $v_0 = 0$ (a vector of zeros). First note that the update at timestep t of the exponential moving average $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (where g_t^2 indicates the elementwise square $g_t \odot g_t$) can be written as a function of the gradients at all previous timesteps:

$$v_t = (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2 \quad (1)$$

We wish to know how $\mathbb{E}[v_t]$, the expected value of the exponential moving average at timestep t , relates to the true second moment $\mathbb{E}[g_t^2]$, so we can correct for the discrepancy between the two. Taking expectations of the left-hand and right-hand sides of eq. (1):

$$\mathbb{E}[v_t] = \mathbb{E} \left[(1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2 \right] \quad (2)$$

$$= \mathbb{E}[g_t^2] \cdot (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} + \zeta \quad (3)$$

$$= \mathbb{E}[g_t^2] \cdot (1 - \beta_2^t) + \zeta \quad (4)$$

where $\zeta = 0$ if the true second moment $\mathbb{E}[g_t^2]$ is stationary; otherwise ζ can be kept small since the exponential decay rate β_1 can (and should) be chosen such that the exponential moving average assigns small weights to gradients too far in the past. What is left is the term $(1 - \beta_2^t)$ which is caused by initializing the running average with zeros. In algorithm 1 we therefore divide by this term to correct the initialization bias.

In case of sparse gradients, for a reliable estimate of the second moment one needs to average over many gradients by choosing a small value of β_2 ; however it is exactly this case of small β_2 where a lack of initialisation bias correction would lead to initial steps that are much larger.

from first-order information. The Sum-of-Functions Optimizer (SFO) (Sohl-Dickstein et al., 2014) is a quasi-Newton method based on minibatches, but (unlike Adam) has memory requirements linear in the number of minibatch partitions of a dataset, which is often infeasible on memory-constrained systems such as a GPU. Like natural gradient descent (NGD) (Amari, 1998), Adam employs a preconditioner that adapts to the geometry of the data, since $\hat{v}_t$ is an approximation to the diagonal of the Fisher information matrix (Pascanu & Bengio, 2013); however, Adam’s preconditioner (like AdaGrad’s) is more conservative in its adaption than vanilla NGD by preconditioning with the square root of the inverse of the diagonal Fisher information matrix approximation.

RMSPProp: An optimization method closely related to Adam is RMSPProp (Tieleman & Hinton, 2012). A version with momentum has sometimes been used (Graves, 2013). There are a few important differences between RMSPProp with momentum and Adam: RMSPProp with momentum generates its parameter updates using a momentum on the rescaled gradient, whereas Adam updates are directly estimated using a running average of first and second moment of the gradient. RMSPProp also lacks a bias-correction term; this matters most in case of a value of β_2 close to 1 (required in case of sparse gradients), since in that case not correcting the bias leads to very large stepsizes and often divergence, as we also empirically demonstrate in section 6.4.

AdaGrad: An algorithm that works well for sparse gradients is AdaGrad (Duchi et al., 2011). Its basic version updates parameters as $\theta_{t+1} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^t g_i^2}$. Note that if we choose β_2 to be infinitesimally close to 1 from below, then $\lim_{\beta_2 \rightarrow 1} \hat{v}_t = t^{-1} \cdot \sum_{i=1}^t g_i^2$. AdaGrad corresponds to a version of Adam with $\beta_1 = 0$, infinitesimal $(1 - \beta_2)$ and a replacement of α by an annealed version $\alpha_t = \alpha \cdot t^{-1/2}$, namely $\theta_t - \alpha \cdot t^{-1/2} \cdot \hat{m}_t / \sqrt{\lim_{\beta_2 \rightarrow 1} \hat{v}_t} = \theta_t - \alpha \cdot t^{-1/2} \cdot g_t / \sqrt{t^{-1} \cdot \sum_{i=1}^t g_i^2} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^t g_i^2}$. Note that this direct correspondence between Adam and Adagrad does not hold when removing the bias-correction terms; without bias correction, like in RMSPProp, a β_2 infinitesimally close to 1 would lead to infinitely large bias, and infinitely large parameter updates.

6 EXPERIMENTS

To empirically evaluate the proposed method, we investigated different popular machine learning models, including logistic regression, multilayer fully connected neural networks and deep convolutional neural networks. Using large models and datasets, we demonstrate Adam can efficiently solve practical deep learning problems.

We use the same parameter initialization when comparing different optimization algorithms. The hyper-parameters, such as learning rate and momentum, are searched over a dense grid and the results are reported using the best hyper-parameter setting.

6.1 EXPERIMENT: LOGISTIC REGRESSION

We evaluate our proposed method on L2-regularized multi-class logistic regression using the MNIST dataset. Logistic regression has a well-studied convex objective, making it suitable for comparison of different optimizers without worrying about local minimum issues. The stepsize α in our logistic regression experiments is adjusted by $1/\sqrt{t}$ decay, namely $\alpha_t = \frac{\alpha}{\sqrt{t}}$ that matches with our theoretical prediction from section 4. The logistic regression classifies the class label directly on the 784 dimension image vectors. We compare Adam to accelerated SGD with Nesterov momentum and Adagrad using minibatch size of 128. According to Figure 1, we found that the Adam yields similar convergence as SGD with momentum and both converge faster than Adagrad.

As discussed in (Duchi et al., 2011), Adagrad can efficiently deal with sparse features and gradients as one of its main theoretical results whereas SGD is low at learning rare features. Adam with $1/\sqrt{t}$ decay on its stepsize should theoretically match the performance of Adagrad. We examine the sparse feature problem using IMDB movie review dataset from (Maas et al., 2011). We pre-process the IMDB movie reviews into bag-of-words (BoW) feature vectors including the first 10,000 most frequent words. The 10,000 dimension BoW feature vector for each review is highly sparse. As suggested in (Wang & Manning, 2013), 50% dropout noise can be applied to the BoW features during

Algorithm 2: *AdaMax*, a variant of Adam based on the infinity norm. See section 7.1 for details. Good default settings for the tested machine learning problems are $\alpha = 0.002$, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. With β_1^t we denote β_1 to the power t . Here, $(\alpha/(1 - \beta_1^t))$ is the learning rate with the bias-correction term for the first moment. All operations on vectors are element-wise.

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1]$: Exponential decay rates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$u_0 \leftarrow 0$ (Initialize the exponentially weighted infinity norm)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$u_t \leftarrow \max(\beta_2 \cdot u_{t-1}, |g_t|)$ (Update the exponentially weighted infinity norm)

$\theta_t \leftarrow \theta_{t-1} - (\alpha/(1 - \beta_1^t)) \cdot m_t/u_t$ (Update parameters)

end while

return θ_t (Resulting parameters)

Note that the decay term is here equivalently parameterised as β_2^p instead of β_2 . Now let $p \rightarrow \infty$, and define $u_t = \lim_{p \rightarrow \infty} (v_t)^{1/p}$, then:

$$u_t = \lim_{p \rightarrow \infty} (v_t)^{1/p} = \lim_{p \rightarrow \infty} \left((1 - \beta_2^p) \sum_{i=1}^t \beta_2^{p(t-i)} \cdot |g_i|^p \right)^{1/p} \quad (8)$$

$$= \lim_{p \rightarrow \infty} (1 - \beta_2^p)^{1/p} \left(\sum_{i=1}^t \beta_2^{p(t-i)} \cdot |g_i|^p \right)^{1/p} \quad (9)$$

$$= \lim_{p \rightarrow \infty} \left(\sum_{i=1}^t \left(\beta_2^{(t-i)} \cdot |g_i| \right)^p \right)^{1/p} \quad (10)$$

$$= \max(\beta_2^{t-1}|g_1|, \beta_2^{t-2}|g_2|, \dots, \beta_2|g_{t-1}|, |g_t|) \quad (11)$$

Which corresponds to the remarkably simple recursive formula:

$$u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|) \quad (12)$$

with initial value $u_0 = 0$. Note that, conveniently enough, we don't need to correct for initialization bias in this case. Also note that the magnitude of parameter updates has a simpler bound with AdaMax than Adam, namely: $|\Delta_t| \leq \alpha$.

7.2 TEMPORAL AVERAGING

Since the last iterate is noisy due to stochastic approximation, better generalization performance is often achieved by averaging. Previously in Moulines & Bach (2011), Polyak-Ruppert averaging (Polyak & Juditsky, 1992; Ruppert, 1988) has been shown to improve the convergence of standard SGD, where $\bar{\theta}_t = \frac{1}{t} \sum_{k=1}^t \theta_k$. Alternatively, an exponential moving average over the parameters can be used, giving higher weight to more recent parameter values. This can be trivially implemented by adding one line to the inner loop of algorithms 1 and 2: $\bar{\theta}_t \leftarrow \beta_2 \cdot \bar{\theta}_{t-1} + (1 - \beta_2)\theta_t$, with $\bar{\theta}_0 = 0$. Initialization bias can again be corrected by the estimator $\hat{\theta}_t = \bar{\theta}_t/(1 - \beta_2^t)$.

8 CONCLUSION

We have introduced a simple and computationally efficient algorithm for gradient-based optimization of stochastic objective functions. Our method is aimed towards machine learning problems with